

بخش اول

مقدمه

در این تمرین ابتدا صحت روش BP را بدون نویز و سپس در حضور نویز بررسی می کنیم. سپس مسئله LASSO را با استفاده از داده های نویزی فرمول بندی می کنیم.

قسمت الف

الگوریتم این قسمت به صورت ریاضی در زیر آمده است:

$$\left\{ \begin{array}{l} \hat{s} = \operatorname{argmin} \|s\|_1 = \sum_{n=1}^N |s_n| \xrightarrow{s_n = s_n^+ + s_n^-} \sum_{n=1}^N s_n^+ + s_n^- \\ s.t. \ x = Ds \rightarrow x = [D \quad -D] \begin{bmatrix} s^+ \\ s^- \end{bmatrix} \xrightarrow{\begin{bmatrix} D & -D \end{bmatrix} = y, \begin{bmatrix} s^+ \\ s^- \end{bmatrix} = y} x = D_1 y \end{array} \right. \Rightarrow \begin{cases} s_n^+ = \max(s_n, 0) \\ s_n^- = \max(-s_n, 0) \end{cases}$$

در انتها فرم این مسئله به صورت زیر در می آید:

$$\left\{ \begin{array}{l} y = \operatorname{argmin} \sum_{i=1}^{2N} y_i \rightarrow \text{Linear Programming} \\ s.t. \ x = D_1 y, \ y_i > 0 \end{array} \right.$$

در نتیجه این قسمت با تابع داخلی `linprog()` بدست می آید. بردار S بدست آمده به صورت زیر است:

$$S = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ -2 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

اگر خطای روش BP را بررسی کنیم می بینیم که خطای الگوریتم BP برابر با 7.6002×10^{-17} می باشد.

قسمت ب

اگر از الگوریتم MP استفاده کنیم، می فهمیم که $N_0 = 4$ می باشد. بردار S نیز به صورت زیر می باشد:

$$S = \begin{bmatrix} -6.9927 \times 10^{-16} \\ 1 \\ 5.3055 \times 10^{-17} \\ 9.7751 \times 10^{-16} \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ -2 \\ 0 \\ 3.0328 \times 10^{-15} \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

مشاهده می شود که نتیجه بدست آمده، مشابه نتیجه بدست آمده از الگوریتم BP می باشد. حال اگر خطای این روش را بررسی کنیم می بینیم که خطای الگوریتم MP برابر با 5.0851×10^{-16} می باشد.

قسمت ج

اگر روش BP را روی مشاهدات نویزی اعمال کنیم، خطای بدست آمده ما از این روش برابر با 0.068456 می باشد. (این خطا، خطای نسبتاً قابل قبولی است). همچنین، بردار S بدست آمده به صورت زیر می باشد:

$$S = \begin{bmatrix} 0 \\ 0.9013 \\ 0.1304 \\ 0 \\ -0.2066 \\ 0 \\ 0 \\ 0 \\ 0 \\ -1.9396 \\ 0.0148 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

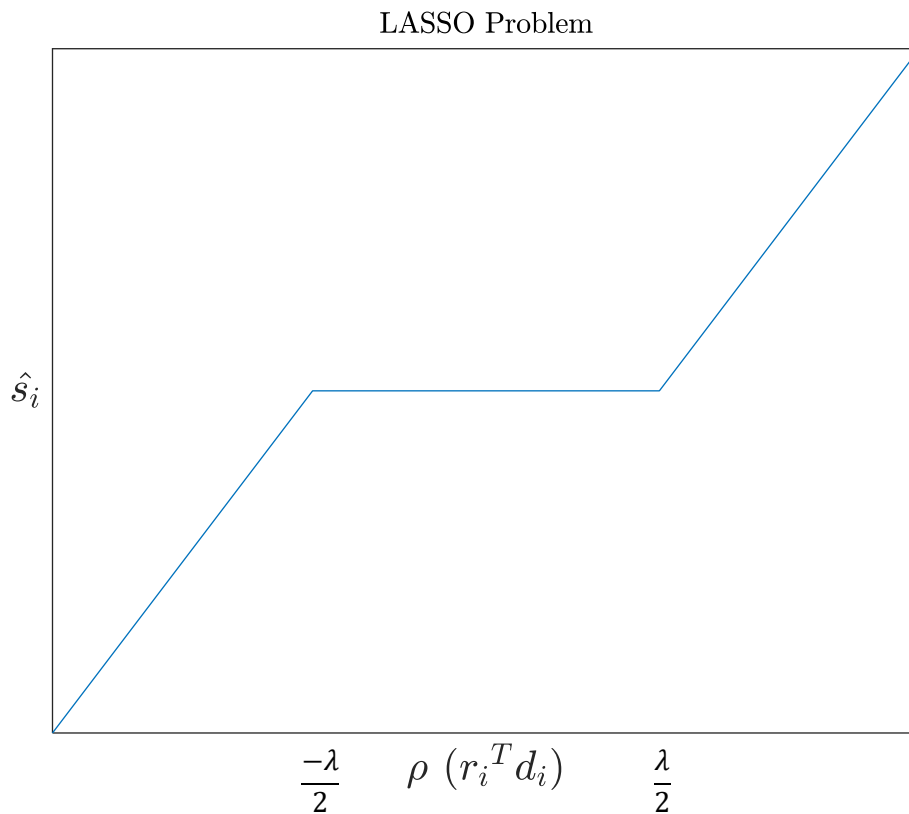
اما منطقاً با خطایی که بدست آوردیم، جواب ما جواب درست نیست. اما حدسی مناسب و با خطای کم می باشد.

قسمت د

برای این قسمت ابتدا الگوریتم LASSO را پیاده سازی می کنیم. این الگوریتم به صورت خلاصه به صورت زیر می باشد:

$$\hat{s}_i = \text{soft}_{\frac{\lambda}{2}}(\rho)$$

شکل نمودار نهایی بدست آمده در مساله LASSO در تصویر ۱.۱ آمده است.



تصویر ۱.۱: تصویر مسئله LASSO

برای این که λ مناسب را بدست آوریم ابتدا مسئله LASSO را روی داده های بدون نویز بررسی می کنیم. با تغییر λ و اجرای کد به دفعات زیاد، متوجه می شویم که λ مناسب برابر با $0.125 = \frac{1}{8}$ می باشد. بردار S بدست آمده از این روش به شکل زیر می باشد:

$$S = \begin{bmatrix} 0 \\ 0.9747 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ -1.9598 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

حال این روش را روی داده های همراه با نویز اعمال می کنیم. مجدداً λ را آنقدر تغییر می دهیم تا به شرایط مطلوبی برسیم.

طبق اجراهای متفاوت، مشاهده می‌شود که λ مناسب برابر با 0.4 می‌باشد. بردار S بدست آمده از این روش به شکل زیر می‌باشد:

$$S = \begin{bmatrix} 0 \\ 0.8916 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ -1.8488 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

خطای مسئله LASSO نیز مانند قسمت قبل و برابر با 0.068456 می‌باشد. می‌دانیم به طور شهودی λ میزان اهمیتی است که به قید Sparsity می‌دهیم. اگر این پارامتر را زیاد کنیم (λ از 2.1 بیشتر شود) دیگر مسئله LASSO تنها به Sparsity اهمیت می‌دهد ($N_0 = 1$ می‌شود) و اگر این پارامتر را کم کنیم (بازه λ از 0.4 تا 0)، روند S بدست آمده به این صورت است که ما Sparsity را از دست می‌دهیم، اما جواب همچنان درست می‌باشد (با N_0 بیشتر). در نتیجه به صورت کلی می‌توان گفت که بهترین بازه برای λ در این مسئله، بازه ای محدود اطراف 0.4 می‌باشد.

همچنین، اگر معیار درستی ما خطا باشد، بازه λ از 0 تا 4.2 می‌باشد. (این بازه، خطای 0.068456 را نتیجه می‌دهد). اگر λ بیرون این بازه باشد، خطای ما برابر با 1 می‌شود. (در واقع آنقدر به قید Sparsity بها داده می‌شود که مسئله $N_0 = 0$ در نظر می‌گیرد).

بخش دوم

مقدمه

در این بخش به دنبال دو روش پیشنهادی برای تولید frame هستیم.

بخش ۱

برای ایجاد یک frame با ابعاد $2 \times N$ ، فرض می‌کنیم که در مختصات دو بعدی قرار داریم. (این فرض، فرض درستی است). سپس، چون مختصات ما دو بعدی است، نتیجه می‌گیریم که $\mu(D)$ هنگامی در کم‌ترین حالت ممکن قرار دارد که بردارهای d بیشترین فاصله را از هم داشته باشند.

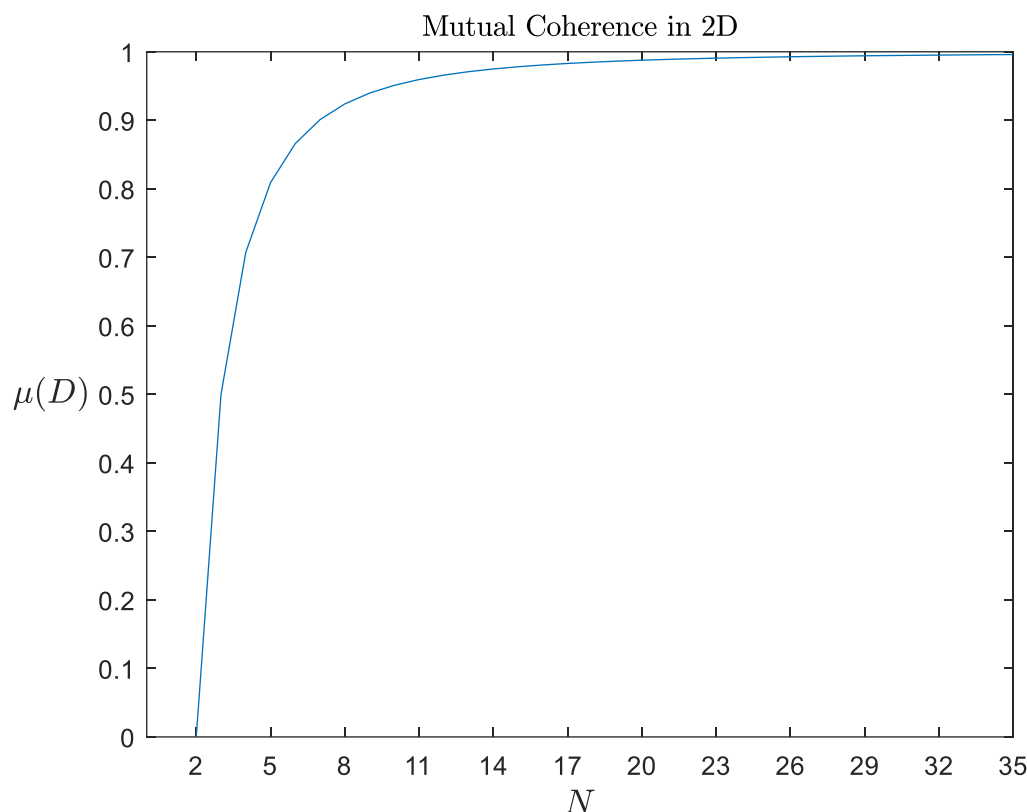
می‌دانیم که هر بردار یکه در صفحه دو بعدی به صورت زیر قابل نمایش می‌باشد:

$$d_i = \begin{bmatrix} \cos(\varphi_i) \\ \sin(\varphi_i) \end{bmatrix}$$

حال مسئله را به دو بخش تفکیک می‌کنیم. (N های زوج و N های فرد). برای N های فرد، اگر مایل باشیم که $\mu(D)$ را کمینه کنیم، کافی است بردارها را در یک صفحه و با بیشترین زاویه از هم قرار دهیم؛ با این کار $\mu(D)$ را کمینه کرده ایم. اما این روش راه مناسبی برای مسئله‌های با N زوج نمی‌باشد؛ زیرا هنگامی که زوج است اگر بردارها را با بیشترین زاویه از هم قرار دهیم، به علت تقارن مسئله همیشه دو بردار وجود خواهند داشت که راستای آن‌ها یکسان، اما در خلاف جهت یکدیگر باشند. بنابراین اگر الگوریتم مناسب برای N های فرد را بر روی N های زوج اعمال کنیم، $\mu(D)$ برابر با 1 خواهد شد.

البته می‌توان برای تمام N های بازه زاویه φ را از 0 تا 180 درجه گرفت تا مسئله را به دو بخش تفکیک نکنیم.

حال N را متغیر و از 2 تا 35 فرض می‌کنیم و نمودار $\mu(D)$ را به ازای N های متفاوت رسم می‌کنیم. این نمودار در تصویر ۲.۱ آمده است.

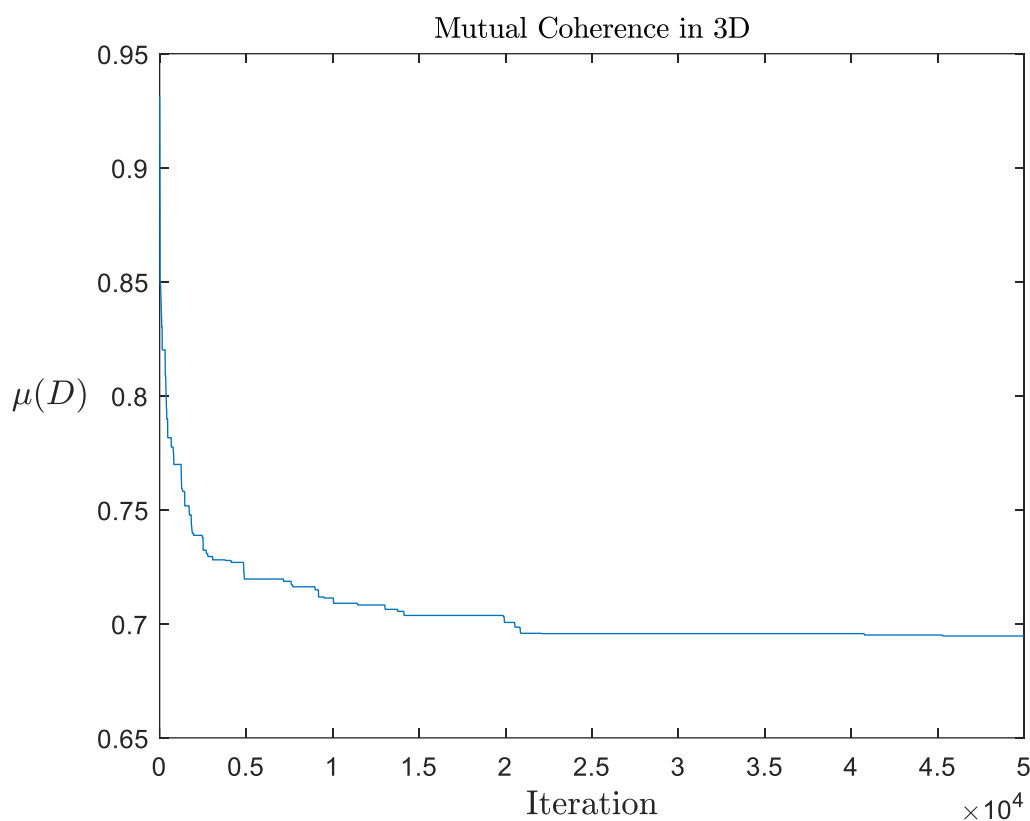


تصویر ۲.۱: نمودار mutual coherence به ازای N های متفاوت

مشاهده می شود که تقریباً در $N = 8$ مقدار $\mu(D)$ نزدیک به 0.9 می باشد. این مقدار بسیار بالا است و مطلوب ما نیست.

قسمت ۲

برای این قسمت، الگوریتم پیشنهادی به این صورت است که ابتدا ماتریس بردار ها (vecs) را به صورت تصادفی تعیین و سپس آن ها را نرمالیزه کنیم؛ سپس با روشی مشابه با Gradient Projection شروع به پیدا کردن جواب مناسب برای این مسئله باشیم. بدین صورت که در یک حلقه for هر بار یک بردار 3×1 تصادفی تولید می کنیم، سپس در یک حلقه for داخلی هر بار این بردار را جایگزین یکی از بردار های vecs می کنیم و $\mu(D)$ جدید را محاسبه می کنیم. سپس آن را با $\mu(D)$ قدیمی مقایسه می کنیم و در صورتی که مقدار جدید کمتر از مقدار قدیمی بود، اقدام به اصلاح ماتریس vecs می پردازیم. با الگوریتم توضیح داده شده، مقدار $\mu(D)$ بدست آمده برابر با 0.6947 می باشد. همچنین، در تصویر ۲.۲ نمودار $\mu(D)$ به ازای تعداد دفعات اجرا رسم شده است.



تصویر ۲.۲: نمودار mutual coherence به ازای تعداد دفعات اجرا

با مقایسه این قسمت با قسمت ۱، می بینیم که با N های نزدیک به هم (تقریباً 10)، در مختصات دو بعدی مقدار mutual coherence ما تقریباً برابر با 0.9 بود؛ در صورتی که در مختصات سه بعدی این مقدار تقریباً به 0.69 کاهش یافت.

بخش سوم

مقدمه

در بخش پایانی نیز می‌خواهیم دو روش MOD و K-SVD را برای جداسازی کور منابع اسپارس بررسی کنیم.

قسمت ۱

می‌دانیم که mutual coherence از رابطه زیر بدست می‌آید:

$$\mu(D) = \max_{i \neq j} |d_i^T d_j|$$

حال با پیاده سازی این فرمول، متوجه می‌شویم که مقدار $\mu(D)$ برای این بخش برابر با 0.868 خواهد بود.

قسمت های ۲ تا ۵

ابتدا روش MOD را به صورت خلاصه تعریف می‌کنیم:

$$\text{Alternation Minimization} \left\{ \begin{array}{l} 1) D \text{ is fixed} \rightarrow \begin{array}{l} x_1 = D s_1 \\ \vdots \\ x_T = D s_T \end{array} \Rightarrow \text{Sparse Recovery Problem} \\ 2) S \text{ is fixed} \rightarrow \begin{cases} \hat{D} = \operatorname{argmin} \|X - DS\|_F^2 \\ \text{s.t. } \|d_n\| = 1 \end{cases} \end{array} \right.$$

لازم به ذکر است که طبق خواسته سوال هر جا که به یک مسئله Sparse Recovery از الگوریتم OMP استفاده می‌کنیم.

نتایج بدست آمده از یک بار اجرا کردن کد برای این الگوریتم به صورت زیر می‌باشد:

• زمان همگرایی: 2.797860 [s]

• Successful Recovery Rate: 82%

حال به توصیف ریاضی روش K-SVD می‌پردازیم:

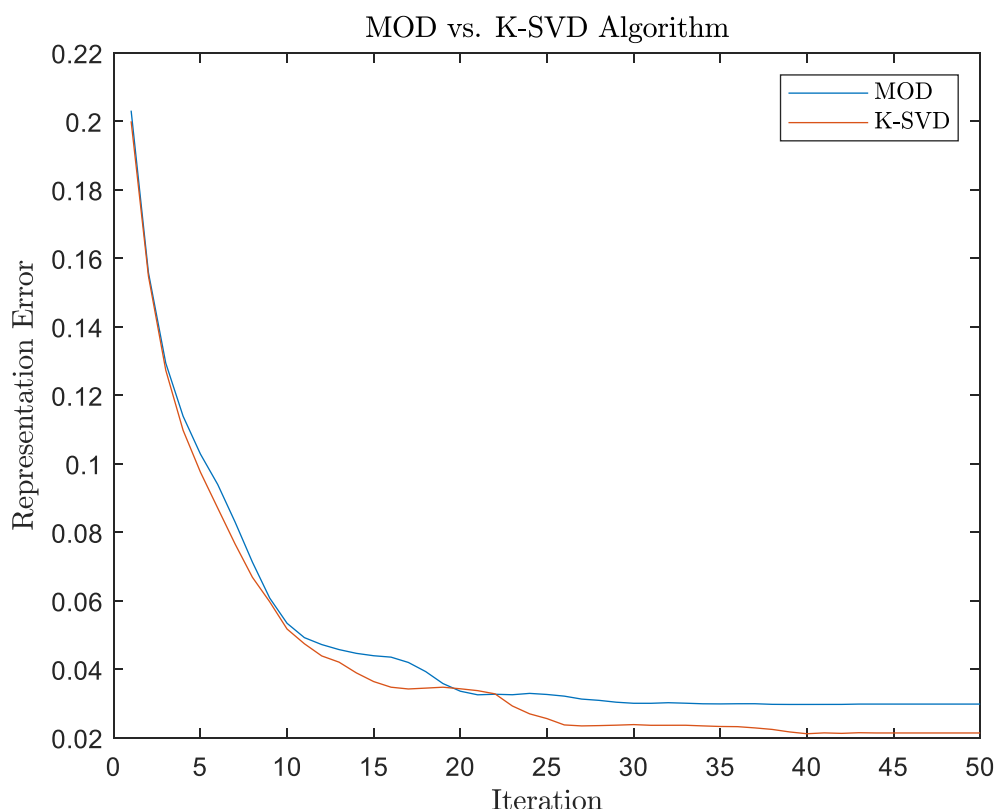
$$\text{Alternation Minimization} \left\{ \begin{array}{l} 1) D \text{ is fixed} \rightarrow \begin{array}{l} x_1 = D s_1 \\ \vdots \\ x_T = D s_T \end{array} \Rightarrow \text{Sparse Recovery Problem} \\ 2) S \text{ is fixed} \rightarrow \begin{cases} \hat{D} = \operatorname{argmin} \|X - DS\|_F^2 \\ \text{s.t. } \|d_n\| = 1 \end{cases} \end{array} \right.$$

نتایج بدست آمده از یک بار اجرا کردن کد برای این الگوریتم به صورت زیر می باشد:

• زمان همگرایی: $3.576926 [s]$

• Successful Recovery Rate: 90%

نمودار همگرایی در تصویر ۳.۱ رسم شده است.



تصویر ۳.۱: مقایسه نمودار همگرایی دو الگوریتم MOD و K-SVD (اجرای اول)

با بررسی تصویر ۳.۱: مشاهده می شود که الگوریتم K-SVD خطای همگرایی کمتری دارد. همچنین مشاهده می شود که روش MOD زودتر همگرا شده است. (زمان همگرایی الگوریتم MOD کمتر است).

لازم به ذکر است که نتایج بدست آمده همیشه مشابه نتایج بدست آمده در بالا نیستند و در بعضی مواقع روش MOD به ما خطای کمتری می دهد. برای مثال نتایج بدست آمده در زیر، نتایج اجرای دوباره کد هستند:

الگوریتم MOD :

• زمان همگرایی: $2.793242 [s]$

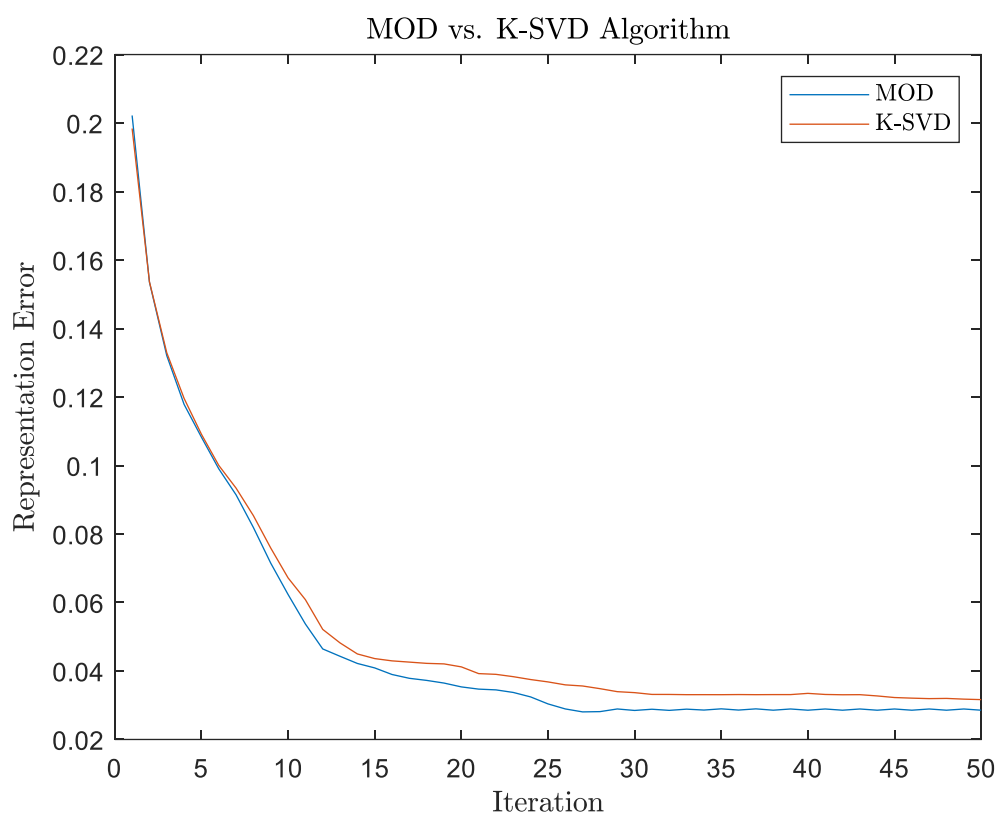
• Successful Recovery Rate: 88%

الگوریتم K-SVD :

• زمان همگرایی: $3.441024 [s]$

- Successful Recovery Rate: 82%

همچنین، نمودار خطای همگرایی بر حسب تعداد دفعات اجرا در تصویر ۳.۲ آمده است.



تصویر ۳.۲: مقایسه نمودار همگرایی دو الگوریتم MOD و K-SVD (اجرای دوم)

قسمت ۶

برای این قسمت، از ماتریس **Correlation** که در قسمت ۳ استفاده کردیم استفاده می‌کنیم. ابتدا روی سطرها ماکزیمم می‌گیریم و اندیس‌ها را پیدا می‌کنیم. سپس ماتریس مرتب شده را توسط اندیس‌های بدست آمده می‌سازیم. در ادامه، از فرمول داده شده در تکلیف استفاده می‌کنیم. این فرمول به صورت زیر می‌باشد:

$$E = \frac{\|\hat{S} - S\|_F^2}{\|S\|_F^2}$$

همچنین، می‌دانیم که نرم فروبنیوس ماتریسی از رابطه $\|S\|_F = \text{trace}(S^T \times S)^{\frac{1}{2}}$ می‌توان استفاده کرد. نتایج خطای این دو الگوریتم به صورت زیر می‌باشد:

الگوریتم MOD :

• خطا: 0.47698

الگوریتم K-SVD :

• خطا: 0.32125